



Maintaining, Monitoring, and Troubleshooting Kubernetes

Implementing Effective Kubernetes Maintenance Strategies

Course Overview

There's a big difference between learning Kubernetes and running it in production. Clusters need upgrading, applications need monitoring, and you have to keep everything reliable while it's live and serving real users. I'm James Willett, and in this course, *Maintaining, Monitoring and Troubleshooting Kubernetes*, you're going to learn how to do exactly that. Let's not waste any time and go over what you're going to learn. First, we're going to tackle cluster maintenance and high availability. You're going to learn how to deploy applications with Helm, the package manager for Kubernetes. You'll see how Helm makes it easy to manage complex deployments with a single command. You'll also learn how to keep applications online during maintenance operations. Then comes the big one, upgrading a live Kubernetes cluster. We're going to upgrade an entire cluster. That's the control plane and all worker nodes, all while deployed applications stay online the entire time. Next, we're going to build a complete observability stack from the ground up. We'll start simple with basic metrics that give us a high-level overview of our cluster and then move to production-grade monitoring with Prometheus and Grafana, the industry-standard tools for getting and viewing metrics. After that, we'll then add centralized logging with Loki, so you can search across all your application logs in one place. And here's where it gets exciting. We're going to use chaos engineering to validate our systems. We'll deliberately inject failures, i.e., latency spikes, errors, and pod crashes, then watch our observability stack catch these issues in real time. Finally, we're going to work through real Kubernetes troubleshooting scenarios, pods that crash repeatedly, images that won't pull, pods stuck in pending, and more. These are the issues you'll encounter in production, and you're going to learn how to diagnose and fix them systematically. Throughout this course, we'll be working with a demo application, a guestbook app with a simple architecture. There's a frontend that serves the web interface, a backend API that handles requests, and a database for storing our application state. This app is intentionally simple. The focus isn't on the application. It's on the Kubernetes concepts. So that's things like deployment strategies, monitoring, and troubleshooting. The app is really just the vehicle for learning those skills. We'll be running this application on a real Kubernetes cluster, one with a control plane and three worker nodes. This is a



production-like environment where you'll see real problems and real solutions. If you want to follow along with the demos and set up your own Kubernetes cluster, there's a full guide on my GitHub that walks you through everything, including deploying the same setup in AWS and installing the tools you'll need. It's completely optional. You can watch and learn, or you can get hands-on practice. Either way works. You're about to learn practical, production-focused Kubernetes skills, maintenance strategies that keep applications online, observability systems that catch issues early, and troubleshooting techniques that fix problems fast. Let's get started.

Introduction to Helm

When you deploy applications to Kubernetes, you're managing multiple manifests. These are often huge metadata files that contain all the details of your application's deployments, services, and configuration. But managing raw YAML manifests quickly becomes a challenge. You're juggling dozens of files, copying and pasting configurations, and making the same change in 10 different places. This is where Helm comes in. Helm is the package manager for Kubernetes. Think of it like apt for Ubuntu or Homebrew for macOS but made especially for Kubernetes applications. Helm helps you define, install, and upgrade even the most complex Kubernetes applications. It's a graduated project in the Cloud Native Computing Foundation maintained by over 400 developers and backed by companies like Google, Microsoft, and Red Hat. It's the industry standard for managing Kubernetes applications. Here's the problem Helm solves. Without Helm, you're managing individual YAML files. Changing any values in those YAML files is difficult, as the value has often been used across multiple different files. With Helm, you package everything into a chart, one cohesive unit. Now you can make whatever changes you need to in one single place, and Helm updates it across all of your YAML files. Another benefit of this is that you can now easily upgrade or roll back a release with a single command. You just need to give the helm upgrade command, and Helm updates all of your YAML files and deploys the new version of your app. These versions of your application are also tracked in Helm, like you would with Git. Let's break down what a Helm chart actually looks like. Every chart follows the same structure, which makes them easy to understand once you've seen a few. First, Chart.yaml. This is the metadata file. It describes what the chart is, what version it is, and who maintains it. This Chart.yaml file is required for every Helm chart. Next, we have one or more values.yaml files. These files contain all of your configuration values, so that's image tags, replica counts, resource limits. Everything that might change between environments goes in here. Finally, the templates directory. This is where your Kubernetes manifests live, but they're not plain YAML anymore. They're Go templates. Helm takes the values from these values.yaml files and injects them into these templates to generate the final Kubernetes manifests. Installing Helm is straightforward. On macOS, it's a simple Homebrew



install. On Linux, there's an official install script. Windows users can use Chocolatey or download the binary directly. Now that you understand what Helm is and how charts work, let's see it in action. In the next clip, we'll deploy our guestbook application using Helm and see these concepts come to life.

Deploying an Application with Helm

Now that we understand what Helm is, let's use it to deploy our guestbook application to the cluster and verify everything is working. First, let's verify our cluster nodes are ready and available. So I've got my control plane here and three separate worker nodes. Just so you know, I've set my kubectl to the k alias just so that I can type these commands quickly. Before we deploy, let's take a quick look at the Helm chart structure. This file here is the Chart.yaml, which contains all of the metadata for the chart. Next, we have the values.yaml file. This is where we define the configuration for our application, including things like how many pods we want to run. After that, we have our template files, which are divided between the back end, the front end, and the database. For this demo, we're actually putting all of the application code in the config maps just to make it easy to deploy straight away to our cluster. The deployment YAML file uses the data from Chart.yaml and values.yaml to generate the final Kubernetes deployment manifests. We also have a pod disruption budget defined, which ensures a minimum number of pods stay available while our nodes are being upgraded. Let me now use Helm to install the guestbook application to our cluster. Let's verify that all our pods are running successfully, and we can see all of our pods are online. So we have four backend pods and six frontend pods. And we also have the Redis pod, which is where our database lives. Let me now set up port forwarding so we can access the frontend service locally. And here's our simple guestbook application running in the browser. So you can see it displays the version number, and we can click this button here to increment the counter. We're also displaying which backend pod handled the request. Let's scale down our application by editing the replica counts in the values file. Now, I'll upgrade the release with Helm to apply these new replica counts. We can see that a new revision of the chart has been deployed. And we can see that we now have two backend pods and four frontend pods. Let's change the replica counts back to six and four, respectively, and then I'll run the same helm upgrade command again. We've now deployed a third revision, and our pods are back to six frontend pods and four backend pods.

Kubernetes High Availability Features

When you deploy applications to Kubernetes in production, uptime isn't optional. It's expected. Your applications need to stay online during routine maintenance, node failures, and cluster upgrades. This is where high availability



features come in. Kubernetes provides three core mechanisms that work together to keep your applications running even when things go wrong. Let's explore each one. The first mechanism is replicas. In the previous demo, we deployed our application with multiple frontend and backend pods. You saw how we could scale those numbers up and down with a simple helm upgrade. But why did we run multiple copies in the first place? The answer is redundancy. Those replicas all serve the same traffic, sitting behind a Kubernetes service that load balances requests across them. Now if one or more pods crashes, maybe due to a bug or maybe due to memory pressure, the other replicas keep serving traffic. Your users don't notice anything. The application stays online. But running multiple replicas isn't enough. During maintenance operations, as you're going to see shortly, you need to make sure Kubernetes doesn't accidentally disrupt all your pods at once. That's where pod distribution budgets come in. A pod distribution budget, or PDP, is a Kubernetes resource that defines how many pods must remain available during voluntary disruptions. Voluntary disruptions are things you control, draining a node for maintenance, upgrading the cluster, or scaling down a deployment. A PDP acts as a safety constraint that tells Kubernetes no matter what, you must keep this minimum number of pods running. Let's visualize exactly how a pod distribution budget protects your application during maintenance. Our application has a total of four pods running across node A and node B. We've defined a PDP rule that tells Kubernetes you must always keep at least three pods available. Now, imagine we need to patch the OS on node A, so we run a node drain command. Without a PDP, the drain command would attempt to evict both pods from node A immediately, which would leave just the two remaining on node B. Because of the pod distribution budget, the eviction API rejects the request to eliminate both pods, and only one is evicted. A new pod then gets created on node B, replacing the one that was destroyed on node A. Once the new pod on node B has started successfully, the other remaining pod on node A can then be destroyed. Since node A is now cordoned off, a fourth pod gets created on node B, bringing us back to our total deployment request of four pods. The PDP ensured that the minimum number of free available pods was never breached. The third piece of high availability is pod anti-affinity. This controls how Kubernetes spreads your pods across the cluster nodes. Without anti-affinity rules, Kubernetes might schedule all your pods on the same node. If that one node fails, your entire application goes down. Pod anti-affinity lets you tell Kubernetes prefer to spread these pods across different nodes. With anti-affinity configured, your pods are distributed evenly, two on each of your three worker nodes. Now if one node fails, you only lose a third of your capacity. The remaining pods on the other nodes continue serving traffic. This is especially valuable during cluster maintenance. When you upgrade a node and reboot it, only the pods on that specific node are affected. The pods on other nodes keep your application running. These three features work together to create true high availability. Replicas give you redundancy, multiple pods handling the same workload. Pod disruption budgets protect you during maintenance, ensuring Kubernetes never disrupts too many pods at once.



And pod anti-affinity distributes your risk, spreading pods across multiple nodes so that no single failure takes down your entire application. Together, these features enable zero downtime cluster upgrades, safe node maintenance, and resilience against failures.

Upgrading and Rolling Back with Helm

Now that we have our application deployed, let's see how Helm makes it easy to upgrade to a new version and roll back if something goes wrong. So all I've changed here is the background color of our application. And in the Chart.yaml, I've changed the app version to version 2. Now, I'll apply these changes using the helm upgrade command. The upgrade was successful and a new revision is deployed. If we take a detailed look at our pods, we can see that they've been distributed between our three different nodes. This is our pod anti-affinity working. We also have our pod disruption budget in place, which ensures that we always have a minimum number of pods available. I'll now set up port forwarding again and jump back over to the application. And if I refresh the browser, we can see that the background changes color, and the version number has also changed. If we use the helm history command, we can see all of the different revisions of our Helm charts. Let's say we wanted to roll back to a different revision. To do that, we can simply use the helm rollback command. This actually deploys a new revision, but it's been rolled back to our previous version 3. If I do the port forwarding again and head back over to the app and if I refresh the browser again, we're back on version 1. Let's now change our app version to 2.1.0 and upgrade our application once more so that it's on this latest version. So we can see here our app is now deployed on version 2.1.0.

Preparing for a Kubernetes Cluster Upgrade

Kubernetes releases new versions every few months, which means cluster upgrades are a regular part of maintaining a production environment. But how do you safely upgrade a cluster whilst keeping all the production applications running on it online? Let's walk through how to plan a safe, zero-downtime cluster upgrade. Kubernetes cluster upgrades normally follow the same order. Control plane first, then worker nodes one at a time. Why this order? Well, the control plane components need to be upgraded before the worker nodes because they're designed to support worker nodes at older versions. So the sequence is upgrade the control plane, then upgrade worker 1, then worker 2, and so on. Upgrade one worker node at a time, never all at once. Let's walk through the pre-upgrade checks that a Kubernetes administrator will always follow. First, verify cluster health. Check that all nodes are ready, all kube-system pods are running, and there are no resource exhaustion warnings. Second, back up the Kubernetes database. The etcd database stores



all your cluster states, every deployment, every service, every config map. If something goes wrong during the upgrade, this backup is your safety net. Finally, review the Kubernetes release notes. Look for braking changes or depreciated APIs that might affect your workloads. Before you touch a production cluster, you need a written maintenance plan. A good maintenance plan should document the entire upgrade sequence. This will include the exact commands you'll run, verification steps at each phase, success criteria, and most importantly, your rollback procedures. Beyond the technical details, there are other things to include as well. Stakeholder communication. Who needs to know about this maintenance window? Timeline and milestones. How long will each phase take? Risk assessment. What could go wrong and how will you handle it? Contact information for escalation if something does go wrong. Now, with the proper high availability features configured that we discussed previously, your applications should experience zero downtime during a cluster upgrade. As you upgrade worker nodes one by one, pods get rescheduled to other nodes. Because you have multiple replicas spread across nodes, your application stays online throughout the process. Now let's talk about what happens if something goes wrong. What if a worker node upgrade fails? You have two main options. Option 1, fix and retry. If the upgrade fails due to a transient issue, maybe a network timeout, or a temporary resource constraint, you can troubleshoot the issue and retry the upgrade on that node. Option 2, replace the node. This is often the safer choice, especially in cloud environments like AWS. The key question is, when do you continue versus when do you abort? If one worker node fails but you can replace it, continue with the upgrade. But if the control plane fails, it's normally better to stop immediately. Just don't upgrade worker nodes until the control plane is stable.

Upgrading a Kubernetes Cluster

Let's work through upgrading our Kubernetes cluster from version 1.33 to 1.34 while keeping the application running the entire time. Let's start by checking the current version of all of our nodes. So everything is currently on version 1.33.6. I'll also verify the Kubernetes API server version. Now, let's confirm our pods are running and distributed across all three work nodes. And let's check the pod disruption budgets are in place that will protect our application during the upgrade. Let me SSH into the control plane node to begin the upgrade. First, I'll create a backup of the etcd database as a safety measure. Let's check the kubeadm version, and we can see that we're on version 1.33.6. We now need to update the package list, and we need to add the Kubernetes 1.34 repository to get access to the new version. I'll update the package list again with the new repository. And now we should see version 1.34.2 is available. Next, we need to unhold kubeadm so it can be upgraded. And now, I'll install kubeadm version 1.34.2. Let's verify that kubeadm has been upgraded, and we can see that it's now on version 1.34.2. We can now hold kubeadm again to prevent automatic



upgrades. Now, let's preview the upgrade plan so that we can see what's going to be changed. So we can see here that the cluster is still running everything on version 1.33.6 at this point, but the target is going to be 1.34.2. If I quickly get the notes, we can see everything is still on 1.33.6. Okay, let's now apply the upgrade to kubeadm. This takes a few minutes to complete. And eventually, we get a success message back. If I now check our guestbook application pods quickly, we can see that our pods have remained active the entire time. Notice that the notes all still show version 1.33 because we haven't upgraded the kubelet yet. But the API server is now running version 1.34.2. Now, let's upgrade the kubelets and kubectl on the control plane. And after the upgrade is done, I'll hold them again to prevent automatic upgrades. I'll now reload the systemd configuration and also restart the kubelet. Now the control plane node shows version 1.34.2. Let's verify that all of the system pods for Kubernetes are healthy. We can see here everything is running, and everything looks to be okay. If we check our application pods, we can see everything looks good here as well. Let's now upgrade our worker nodes, starting with worker 1. I'll cordon worker 1 to prevent new pods from being scheduled on it. Notice that worker 1 is now marked as scheduling disabled. Now, I'll drain worker 1 to safely evict all pods. We can see that the pods have been moved to the other two work nodes. Let's quickly verify that the application is still working during this maintenance. And everything looks good. Our counter still persists, and I can still increment the counter. Now, I'll SSH into worker 1 to perform the upgrade. I'll add the Kubernetes 1.34 repository on this work node and update the package list and install the upgrade for kubeadm. Next, I need to install kubelet and kubectl versions 1.34.2, then reload the system configuration, and again, restart the kubelet service. Worker 1 now shows version 1.34.2. And now, we can uncordon worker 1 so that pods can be scheduled on there again. I'm now going to skip the video ahead and follow the same process for workers 2 and 3. So I've upgraded both workers 2 and 3. If I now get the nodes again, we can see all of our nodes are now running on version 1.34.2. And if we check our application pods, we can see that everything is still running successfully. That's how to upgrade a Kubernetes control plane and its worker nodes.

Establishing Comprehensive Kubernetes Monitoring and Observability

Kubernetes Observability

You can't improve what you can't measure. If you're running a production cluster without observability, you're flying blind. That's not acceptable. So let's talk about observability. When discussing observability, there are three main pillars we're concerned with. Firstly, metrics. These are numerical measurements over



time, things like CPU usage, memory consumption, request rates, error rates. Next, logs. These are timestamped records of discrete events. Think application logs, error messages, and audit trails. Logs tell you what's happened and provide context for when you're troubleshooting issues. Finally, traces. These show the path of a request as it flows through your distributed system. In this module, we're going to focus on the first two, metrics and logs. These are the foundations of Kubernetes observability and where you'll get the most value for your effort. So why invest time and resources in observability? Four reasons. First, you can detect issues before your users complain. If you see memory usage climbing steadily, you can act before pods start getting killed. Second, you understand system behavior under load. When traffic increases, what happens to your CPU usage? What happens to response times? Third, you make informed decisions about scaling and upgrades. Should you add more nodes? Do you need to increase pod replicas? Which pods are consuming the most resources? Fourth, you prove reliability to stakeholders. When someone asks, how reliable is our application, you need data, not guesses. Metrics like uptime percentage, error rates, and response times let you answer with confidence. In this module, we're going to build observability from the ground up. We'll start simple with the Kubernetes metrics server. This gives you basic resource metrics, but has serious limitations. Then we'll move to production-grade tools, Prometheus for metrics, Loki for logs, and Grafana for visualizing everything. Finally, we'll use that data to track reliability with service-level objectives. This is where observability translates into business value.

Kubernetes Metrics Server

To begin our observability journey, let's install the metrics server and explore basic cluster metrics. First, I'll add the metrics-server Helm repository. Now, I'll update the Helm repositories to get the latest chart information. Let me install the metrics server into the kube-system namespace. We get this message back from Helm to say that the metrics server has been installed successfully. I'll now verify that the metrics-server pod is running, and we can see the metrics-server pod here. What we can do now is we can view CPU and memory metrics for all of our nodes. So I can simply type `k top nodes`, and we can see the CPU cores, the CPU percentage, the memory in bytes, and the memory percentage of the control plane and each of the worker nodes. What we can also do with metrics server is that we can see the resource usage for each of our pods. So if I type `k top pods`, we can see the CPU cores and the memory in bytes for each of our pods. So these are obviously very basic and high-level metrics, but it does give us a high-level overview of our control plane and our worker nodes.

Introduction to Prometheus and Grafana



For production Kubernetes environments, you need more than basic metrics. You need historical data, custom metrics, powerful querying, and visualization. That's where Prometheus and Grafana come in. These are the industry standard tools for Kubernetes observability. Let's look at what each one does. Prometheus is an open-source monitoring and alerting toolkit built specifically for metrics. At its core, it's a time series database that stores numerical data points over time. For example, CPU usage and memory usage. The great thing about Prometheus is that it scrapes metrics from your cluster and applications automatically. You don't have to push data to it. Prometheus also gives PromQL, the Prometheus Query Language, which lets you analyze your metrics in sophisticated ways. Finally, Prometheus has built-in alerting. You define rules like alert me if CPU usage exceeds 80% for 5 minutes, and Prometheus monitors these conditions and fires alerts when thresholds are breached. While Prometheus stores and queries metrics, Grafana visualizes them. Grafana is an open-source visualization and analytics platform that connects to multiple data sources like Prometheus and turns that data into dashboards with graphs, gauges, and tables. The Grafana community maintains hundreds of pre-built dashboards that you can import. They show things like cluster health, node resources, and pod statistics. You can also create custom dashboards for your applications. For example, to track your API request rates or your database query latency. Grafana can also combine data from multiple sources into a single dashboard. You might have Prometheus for your Kubernetes metrics, your cloud provider's monitoring service for infrastructure metrics, and your application database for business metrics, all visualized together. With Prometheus and Grafana, you can monitor metrics at four different levels. Cluster-level metrics show overall cluster health. This is things like total CPU usage across all nodes, total memory consumption, and network throughput. Node-level metrics show individual server resources. That's things like per-node CPU usage, per-node memory, and disk utilization. This helps you identify hotspots, i.e., which nodes are overloaded. Pod-level metrics show container resources. So that's things like per-pod CPU, memory usage, restart counts, and container states. This helps you identify which workloads are resource-hungry and which pods are unstable. Finally, application-level metrics are custom metrics from your application, things like HTTP request rates, error rates, and database query counts. This is where observability connects to business value. Prometheus and Grafana together give you production-grade observability. Let's get them installed on our Kubernetes cluster next.

Detailed Metrics with Prometheus and Grafana

Now, let's install Prometheus and Grafana and explore metrics at different levels. First, I'll add the prometheus-community Helm repository, and I'll update my list of Helm repositories. Let me create a dedicated monitoring namespace. Now, I'll install the kube-prometheus-stack, which includes



Prometheus, Grafana, and all the necessary exporters. With the stack installed, let me now retrieve the Grafana admin password from the secret. And now, I'll set up port forwarding to access Grafana. Let me copy the secret to my clipboard, and I'll open my browser on localhost port 3000. The username we can log in with is admin, and then we just paste in the password that we copied previously. Inside Grafana, let's take a look at our connections. I'll look at the configured data sources, and we can see that we have Prometheus set here already. If I now go over to the dashboards, we can see that our installation of Grafana comes with many preconfigured dashboards that we can choose from. Let's take a look at the Compute Resources / Cluster dashboard, and we can see various metrics here for our cluster. Let's take a look at one for our nodes. And again, more metrics for each of the nodes running in our cluster. Finally, we can also take a look at a dashboard for our specific pods for our guestbook application. So we can choose the individual pod that we want to look at the stats for. So back in our application code, I've added in some Prometheus metrics into our application. These are things like the request counts, the counter increments, and the request duration. Now, let's upgrade our guestbook application with Prometheus metrics enabled. We can check the Helm history to see that the new chart was applied, and I'll verify that all the pods are up and running. A few pods are still terminating, but that's because they're just updating to the new version. And let's check that the service monitor was created for Prometheus to discover our metrics. Let's take a quick look in the browser at what our metrics look like from Prometheus. I'm going to port forward the backend metrics endpoint so we can see those raw metrics. And if I jump over to my browser at localhost 5000, I can see all of the raw metrics being exposed by our application. Now, let's access the Prometheus UI to query these metrics directly. So to do that, I'm going to port forward to the Prometheus UI at port 1990. Back in the browser again, we can now access the Prometheus UI. Let me demonstrate a few example queries in Prometheus. So for example, we could look for something like the instance node CPU ratio. And because we've added in custom metrics, we can actually search for things like guestbook and see the various different metrics that we've defined in our application code. Back in Grafana, let's now create a custom dashboard to visualize our application metrics coming from Prometheus. So I'll click on New dashboard and then Add visualization. I'll make sure I choose Prometheus as the data source. And I'm going to add a panel to show requests per second. I just need to go down here to the part where I type in the query, and I can use this query to get the guestbook requests total over a 1-minute interval. And that's what the visualization looks like in our dashboard. So I can now save this dashboard, and we can use it for monitoring.

Centralized Logging in Kubernetes

Metrics tell you what is happening in your system, but they don't tell you why. That's where logs come in. Every application produces logs. But in



Kubernetes, accessing and using those logs effectively is more challenging than you might expect. The basic way to view logs in Kubernetes is `kubectl logs`. But this approach has serious limitations in production. First, you can only view logs from one pod at a time. If you have 10 pods and you're troubleshooting an intermittent issue, you're running `kubectl logs` 10 times. Second, logs are lost when pods restart or crash. If a pod crashes due to an error, the logs that will tell you why often disappear with the pod. Third, there's no way to search across multiple pods. You can't ask, show me all the error logs from any backend pod in the last hour. You have to check each pod individually. Fourth, there's no retention. They exist only as long as the pod exists. And even then, they're rotated and discarded to save disk space. Finally, there's no way to correlate logs with metrics. When you see a metric spike in Grafana, you can't easily jump to the relevant logs to see what was happening at that exact moment. The solution is centralized logging with Loki and Grafana Alloy. Loki is a log aggregation system. It collects logs from your entire cluster and stores them in a central location with retention. You can search across all logs, filter by pod or namespace or log level, and keep logs for days or weeks. Grafana Alloy is a daemon set that runs on every node in your cluster. It collects logs from all containers on that node and ships them to Loki. Right now, our application uses basic print statements that output plain text. We're going to upgrade that to structured logging using JSON format with fields like timestamp, log level, event type, and custom data. This makes logs much easier to query. For example, you can filter to show only error-level logs. You'll also be able to extract specific values from the log fields. Centralized logging with Loki and Grafana Alloy solves the limitation of `kubectl logs`. You get persistent storage, cross-pod searching, powerful querying with LogQL, and integration with your metrics dashboard. Let's see it in action next.

Logging with Loki and Grafana Alloy

Now, let's install Loki and Grafana Alloy to centralize our logs and integrate them with our metrics. First, I'll add the grafana Helm repository. I'll update the Helm repositories list. And now, let me install Loki using our custom configuration. After that's done, we can next install Grafana Alloy, which collects metrics from our pods and sends them to Loki. And again, we're using a custom values file that contains all of our configuration. Let's verify that both Loki and Alloy pods are up and running. And now, let's upgrade our guestbook application with structured logging enabled. In order to be sure that we have the latest version of the backend pods running, I'm going to do a rollout restart of our deployment. I'll do the same thing for the frontend pods as well. Let me now check that all of our application pods are running. And if I head over to the browser quickly, I can see that our application is running and working as normal. So I'm going to click the increment counter button a few times just to generate some logs for our application. Let's head back over to Grafana and add a new connection for



Loki. So I'll go to add a new connection. I'll search for Loki. Click on the Loki data source. And then I'll click Add new data source. I need to provide the connection URL. And for our cluster, it's loki-gateway port 80. I'll now scroll down to the bottom and click on Save & test, and we get a message back saying that the data source was successfully connected. Let's now see how we can view the output from Loki in Grafana. So we're going to go over to the Explore tab. I need to make sure that I change the data source to be Loki. And let's try a query to get all of the logs from our back end. So if I execute this query, we get a graph here to show our log volume, and we can see all of our logs from our back end are now appearing in Grafana. And those logs are nicely structured with things like the timestamp, the level, the message, the pod, etc. We've also set up our application so that we can look at certain events. So let's look for the button clicked event, and then we get all of the logs related to whenever a button is clicked in our application. Let's now put some of these Loki logs into a dashboard. So I'm going to go back to the Dashboards tab, click create dashboard, and click add a new visualization. I need to choose Loki as the data source, and I want to create a dashboard visualization that just shows our logs. I'll just give it a name of Counter incremented logs, and I'll put in the same query that we just used on the Explore tab. Let me click Run query, and we can now see that all of our logs appear in this visualization. Let me now save this dashboard so that it persists, and I'll just give it a name of Guestbook Dashboard. Let's now go ahead and add in one more visualization. I'll make sure that I choose Loki as the data source. And this time, I'm going to use the query that we had for our button clicked events. I'll now run the query. And we can see that our visualization gets populated with those events. I'll just scroll down here on the right-hand side and change this to a bar graph. That looks good, so let me save the changes to this dashboard. And now we have a nice dashboard that displays both our logs and our visualization with the button clicks side by side.

Service Level Indicators and Objectives

We've talked about collecting metrics and logs. We've built dashboards that visualize that data. But how do you know if your system is actually reliable? I mean, how do you measure success? This is where service level indicators, service level objectives, and service level agreements come in. Let's break down what each of these means. A service level indicator, or SLI, is a specific metric that measures one aspect of your service's reliability. There are three common types of SLIs. Availability measures the percentage of time your system is reachable and ready to serve traffic. It's calculated as uptime divided by total time. Latency measures response time, typically as a percentile. P95 latency means 95% of requests are faster than this threshold. Error rate measures the percentage of requests that fail. It's calculated as errors divided by total requests. A service level objective, or SLO, is your target for an SLI. It's the internal goal your team commits to. For example, your availability SLO might be



99.5% uptime. Your latency SLO might be P95, less than 200 ms. Your error rate SLO might be less. 0.5% errors. These are all internal commitments. They are targets that guide your operational decisions and help you prioritize reliability work. SLOs are not aspirational goals like be as fast as possible. They're specific, measurable targets with numbers attached. A service level agreement, or SLA, takes an SLO and adds consequences. It's a contract with your customers. For example, we guarantee 99.5% uptime. And if we violate that, you get a 10% service credit. That's an SLA. It's more formal and legally binding than an SLO. Not every SLO needs to be an SLA. You might have internal SLOs that are stricter than your external SLAs. For example, you might target 99.9% internally, so you have buffer room before violating your 99.5% SLA with customers. Now there's one concept that brings this together, error budgets. If your availability SLO is 99.5%, that means you have a 0.5% budget for failures. When your error budget is low, for example, when you've used up most of your allowed downtime, then that's when you'll typically take action. You'd probably focus more on reliability for a while or slow down deployments to production. Error budgets help you balance velocity and reliability. They give you a data-driven way to make decisions about risk.

SLO Tracking and Chaos Engineering

In this demo, we're going to build an SLO dashboard in Grafana, and then we're going to use chaos engineering to deliberately violate each SLO. So I've added this code into our backend application to inject different types of chaos. We can inject random amounts of latency, or we can inject random amounts of errors, and we can even simulate our pods having to crash. Let me now upgrade the guestbook application with chaos engineering capabilities enabled. I'm also going to do a rolled-out restart on both the front end and the back end to be sure that all of the new pods have been deployed. Now I've already created an SLO dashboard that we can use for this demo. You can find the dashboard here, and I'm just going to import this into Grafana. So back in Grafana on the Dashboards page, I'm going to click New and then Import. And then I'm going to search for that SLO dashboard file. And this is our SLO dashboard that tracks availability, latency, and error rates in our application. Now, in order to trigger chaos in our application, we need to make a few API calls to our back end. So I'm going to enable port forwarding on our backend API at port 5000. Let me first send this API call just to check the status of our chaos. So we can see here that chaos is currently not enabled. Now, let's enable latency chaos to violate our latency SLO. I'm going to generate some traffic in our application by clicking the Increment Counter button several times. And if we head back over to our Grafana dashboard, after a minute or so, our dashboard picks up those logs and it updates appropriately. So we can see our latency has now increased to 392 ms, and we can also see by looking at the graph the impact on our SLO target for latency. And if we scroll down a bit, our dashboard also displays all of the logs related to the



latency that we just injected. Let me go back to the terminal and send another API request. But this time, it's going to disable the chaos again. Now, let's enable error chaos to violate our error rate SLO. And I'm going to go back to the application and click on the Increment Counter button a few times just to generate some more traffic. Notice that when we're clicking on this button, we're occasionally getting an undefined message, which suggests that the errors are appearing in our application. I'll head back over to the Grafana dashboard, and we can see after a few seconds, the impact of the errors on our error rate SLO. Again, if we scroll down to the logs, we can see all of these errors now appearing in our logs as well. Let me go back over to the terminal and disable the chaos again. And for the final scenario, we're going to try and crash some of our backend pods. So let me start watching the backend pods in real time just so that we can see this clearly. So here, I'm now watching all of the backend pods, and these will update each time one of the pods status changes. Now, I'll enable crash chaos, which will cause pods to crash and violate our availability SLO. I'll head back over to the application and click on the Increment Counter button a few times. We can see that we're getting errors occasionally that the back end is unavailable. And eventually, we're getting no visits count at all because all of the backend pods have crashed. Back in the terminal, we can see where the backend pods have crashed with an error. If I head over to the Grafana dashboard, we can see the logs where our backend pods have been crashing. And if we go back up, we can see the impact on our availability SLO. To finish the demo, let me go back to the terminal and disable chaos one final time.

Diagnosing and Resolving Common Kubernetes Issues

Kubernetes Troubleshooting Workflow

Things break in Kubernetes. Pods crash, images won't pull, and services can't connect. This is all perfectly normal. The difference between a good Kubernetes operator and a struggling one isn't where the problems happen; it's how quickly and systematically you can diagnose and fix these issues. So in this module, we're going to work through five common Kubernetes issues you'll encounter in production. First, `CrashLoopBackOff`, when pods keep starting and crashing repeatedly. Second, `ImagePullBackOff`, when Kubernetes can't pull your container image. Third, pending pods, when pods are stuck and won't schedule onto nodes. Fourth, networking issues, when services exist but can't communicate. Fifth, `OutOfMemoryKilled`, when containers exceed their memory limits and get terminated. Each of these has different symptoms and different root causes, but we'll be using the same troubleshooting workflow for all of them. Here's the workflow you'll follow for every issue. Step 1, observe. Run



kubectl get pods. What's the symptom? Which pods are affected? Step 2, describe. Run kubectl describe pod. Get the detailed status and event history. Step 3, logs. Run kubectl logs. Check what the application is actually reporting. Step 4, events. Check cluster events or specific resources. Look for error messages from Kubernetes itself. Step 5, fix. Make the change. Update your configuration, fix your manifest, and adjust your resources. Step 6, verify. Confirm the fix worked. Pods should be running and the application should be healthy. This workflow becomes second nature with practice. The key is staying calm, being systematic, and letting kubectl guide you to the problem. Let's start troubleshooting. First up, CrashLoopBackOff, one of the most common pod errors you'll see in Kubernetes.

Troubleshooting CrashLoopBackOff

Now, let's put our troubleshooting workflow into practice with our first common issue, CrashLoopBackOff. Let me check the pod status, and we can see that one of our pods is in a CrashLoopBackOff state. So let's first describe the pod to see the crash counts and the exit code. So you can see here that we've got some event logs, but there's nothing too obvious here suggesting why the CrashLoopBackOff is happening. So let's try and check the logs from the previous crashed container to see what went wrong. And we can see that we've got some clear error messages here to suggest that there's a problem with Redis. Let's take a look at our Helm guestbook values to see if we can see what the problem is. And sure enough, we can see that we've set the Redis host to a Redis service that does not exist. Let's check our list of services to find what the correct Redis host name should be. So it should be guestbook-redis. And let's take a look at the scenario file where I injected this error. And here, you can see where I deliberately set the Redis host to be Redis service does not exist to simulate this error message. In order to fix the issue, I'm just going to upgrade back to the correct values file. So I'll just do helm upgrade and point back to the original values file. And after a minute or so, if I now get the pods again, I can see that the pods are now starting to spin up successfully.

Troubleshooting ImagePullBackOff

Now, let's tackle another frequent pod lifecycle error, ImagePullBackOff. To simulate this error, I've deployed the application with this ImagePullBackOff values file. So let's start as always by checking the status of the pods, and we can see we've got this ImagePullBackOff error. Let's first describe this particular pod to see if we can get more details on the error. And straightaway, we can spot the problem. We're trying to pull this image that doesn't exist. Let me check the Helm values to see what image configuration was deployed. And sure enough, we can see here that we have this tag, 99.99.99-nonexistent. So we have a nonexistent



tag for our Python. And if I show you the values file, you can see here this is where we set that tag. So as always to fix this, I'm just going to upgrade back to the original version with the original values. Once that's deployed, I can get the pods again, and we can see that our ImagePullBackOff error has now disappeared.

Troubleshooting Pending Pods

Next up, let's troubleshoot pods stuck in pending state. So again, I've deployed our application with this pending-values.yaml file. Let me first check the pod status, and we can see this pod here is stuck in a pending state. Let's describe the pod and see if we can get any insights into why the scheduler can't enable it. That's interesting. We have an error message at the bottom saying that there's insufficient CPU. Let me check the resources spec for this particular pod. Ah, that's interesting. We can see here that we're asking for eight CPUs. Let me check the available CPU on each node using `k describe nodes`. I can see from these logs that each CPU only has around two cores per node. I can also use the `top nodes` command to see node's capacity. So the problem is that none of our nodes have capacity to schedule this pod. If we take a look at the pending-values.yaml file, we can see here this is where we were trying to request eight CPUs, but our nodes only have around two CPUs each. As always, to fix this, I'll just upgrade our application back to the original values. After the application's been deployed, I'll do `k get pods` again, and we can see all of our pods are now back up and running.

Troubleshooting Networking Issues

Now, let's shift gears and troubleshoot a networking issue where services aren't connecting properly. For this demo, I've deployed our application with the networking-values.yaml. Let me first check the pods. And on first glance, it seems that our application is running normally. But if I open up our application in the browser, we can see that we have this error message saying that the backend is unavailable. We're also unable to click the Increment Counter button. Let's check our endpoints and see if we can spot anything wrong. That's interesting. It looks like nothing's being set for the backend endpoint. Let me describe the backend service to see what its selector is. Aha, and we can see that the selector has been set to the wrong app name. Let's check the pod labels of the back end to see if they match the service selector. Here's the issue, the service selector and the pod labels don't match. The labels in our backend are looking for the guestbook backend app. Let me check the Helm values to confirm the misconfiguration. And sure enough, the service selector app is set to the wrong app name. If we take a look at our networking-values.yaml, we can see here that we have this selector with the wrong app name. As always, to fix this, I just need to redeploy with the original values. If I then head back over to the application, we can see that it's



now back up and running successfully.

Troubleshooting OOMKilled

Final troubleshooting scenario is OutOfMemoryKilled, which happens when containers exceed their memory limits. I've deployed our application with this oomemorykilled-values file. Let me check our pods, and I can already see that one of the pods has had three restarts. Let me check them again, and I can now see that this pod is status OOMKilled. Let's describe that pod to see the OutOfMemoryKilled termination reason and the exit code. So if I scroll up here, I can see this reason here, CrashLoopBackOff, OutOfMemoryKilled. Let's see if we can get any logs for the pod. Hmm, okay, nothing's coming back. Let's take a look at the memory limits that are configured for this particular pod. And that's interesting. We can see that the pod is set to only 32 Mi. Let me get the Helm values to confirm. And yes, sure enough, we're setting the memory to 32 Mi in the back end, which is going to be nowhere near enough for our application. If I just check the oom-killed-values file, we can see here this is where we were setting the memory to 32 Mi, which is causing the problem in our application. So as always, to fix this, I'll just redeploy with the original values. And then after a minute or so, I'll check our pods, and we can see all of our pods are now back up and running. We've just worked through five of the most common Kubernetes issues you'll encounter. The key to all of this is stay systematic, use kubectl describe and logs, and follow the troubleshooting workflow that was outlined. Thank you for taking this course on maintaining, monitoring, and troubleshooting Kubernetes. If you have any questions or thoughts, please put them in the discussion board. I'd love to hear from you. I'm James Willett, and I look forward to seeing you in another course soon.